

SAÉ 2.02 – Exploration algorithmique d'un problème

Rapport de projet

Maxime SCHORSCH-LEGRAND et Alexandre BESSON (Groupe S2 D)

27/05/2026

Sommaire

1. Présentation du projet	2
2. Représentation d'un graphe	2
3. Calcul du plus court chemin via l'algorithme du point fixe	2
4. Calcul du plus court chemin via l'algorithme de Dijkstra	3
5. Application aux données du réseau STAN	3
5.1 Comparaison des temps de calcul des algorithmes de Bellman-Ford et Dijkstra	6
5.2 Comparaison des chemins obtenus par notre application et le moteur de recherche du réseau STAN	4
6. Conclusion	

- Nom/Prénom/Groupe des membres du binôme :
 - o SCHORSCH-LEGRAND Maxime(Groupe S2D)
 - o BESSON Alexandre (Groupe S2D)

URL du dépôt git :

https://github.com/Alexandre-Besson/SAE_02_ExploAlgo_BESSION_Alexandre-SCHORSCH-LEGRAND_Maxime.git

Nos classes main :

MainBellmanFord.java

MainDijkstra.java

MainTransport.java

commande pour compiler en jar :

MainBellmanFord:

jar cfe bellman.jar MainBellmanFord saeobj\out\production\saeobj*.class

MainDijkstra:

```
jar cfe dijkstra.jar MainDijkstra saeobj\out\production\saeobj\*.class
```

MainTransport:

```
jar cfe sae.jar MainTransport saeobj\out\production\saeobj\*.class
```

Présentation du projet :

Notre application a pour but de donner le chemin le plus court entre deux point en utilisant la méthode de bellmanford et de dijkstra

la méthode maintransport prends 2 paramètre le point de départ et celui d'arrivé quand on exécute maintransport elle va lancer les classe bellmanford et dijkstra et donne leur temps pour chaque méthode

Test bellman:

vérifie avec un chemin donné que la longueur soit correcte et quelle point sont adjacents

test dijkstra : vérifie avec un chemin donné que la longueur soit correcte

2 Représentation d'un graphe :

Nous utilisons des liste pour permettre au code de faire les calcul

Nous avons le droit de récupérer les stations et leur adjacents pour remplir les liste

Les graphes ont été basé sur un modèle donné dans l'énoncé de la Sae

Nous avons 3 Test un pour tester l'ajout et récupérer les valeurs d'une liste d'arcs et de adjacents, un autre pour tester l'affichage et un dernier pour tester si un noeud n'existe pas

3 Calcul du plus court chemin via l'algorithme du point fixe

Nous utilisons des liste pour permettre au code de faire les calcul et les valeur de chaque noeuds

Au fil du programme les parents et les valeurs des noeuds changent
Le programme a pour base l'algorithme de math de Bellman Ford

Nous avons un grand test pour les valeur incorrect ou parents incorrect

4. Calcul du plus court chemin via l'algorithme de Dijkstra

Nous utilisons des liste pour permettre au code de faire les calcul et les valeur de chaque noeuds

Au fil du programme les parents et les valeurs des noeuds changent

Le programme a pour base l'algorithme de math de Dijkstra

test que le programme trouve bien le bon chemin avec des informations donné

5. Application aux données du réseau STAN

Nous utilisons un bufferedReader et filereader pour récupérer les différentes stations

Nous appelons les classes pour les utiliser sur les donnés récupéré

Pour notre nous avons un chemin aléatoire sans réel importance

départ	arrivée	chemin	temps calcul Bellman-Ford	temps calcul Dijkstra
MXAME0	ESKLB1			
chemin:				
MXAME0;MXMRT1;MXDVL0;NYECO1;NYVAY1;MLLIO2;MLMAI0;MLBAR0;MLJER0;MLALE1;MAMNV1;MAPRC1;MAPIC6;MALRO0;MACPS2;ESCES1;ESMZM9;ESMET0;ESKLB1				
bellmanford : 54852700 ns				
dijkstra : 30733900 ns				

Conclusion :

Nombre de classe :16

Nombre de méthodes : 18 sans les classes main ou les test

Nombre de test : 4

En moyenne Dijkstra est le plus rapide par 10000000 ns

Notre programme demande qu'on lui donne la liste avec toute les informations , on pourrait l'améliorer pour qu'il récupère directement depuis une base de donnée qui est mise à jour

L'utilisation de Python qui à été difficile à utiliser car selon l'ordinateur nous avons des problèmes juste pour lancer les jar alors que l'autre était capable de les lancer sans problèmes